Cyntexa Jaipur Hiring Drive

Coding Rounds — Detailed Analysis + 100 High-Probability Practice Questions

Prepared for the two-coding-round campus-drive format
Evidence base: recent Cyntexa Jaipur/Sitapura interview reports from 2025–2026, including reported Associate Software Engineer / Associate Software Developer experiences.

Important: The 100-question section is a probability-weighted preparation set, not a leaked question paper. Exact future questions can change.

Document date: 16 August 2026

1. Executive Summary

Recent reports show that Cyntexa's Jaipur hiring process can vary by role and hiring cycle. Some 2025–2026 reports describe two coding rounds; others describe three DSA rounds before HR. For the specific two-round format, the most recent detailed reports show a basic/easy-medium round followed by a longer problem-solving round with strings, arrays, patterns, matrices and optimization.

Strongest recurring signal: arrays + strings + pattern printing + 2D matrices. HashMap/frequency logic, two pointers, subarray logic, palindrome/string parsing, sorting/merging, and basic optimization also recur.

What this means for your campus drive: do not spend most of your time on advanced graphs or hard DP. First become extremely fast at array/string implementation and pattern/matrix questions, then add medium problems that combine two techniques.

Probability guide used in this document

Priority	Meaning	Examples
P1 — Very High	Directly reported or extremely close variant	Compression, merge arrays, palindrome, equilibrium, spiral, transpose
P2 — High	Same concept family with a small twist	Two Sum, frequency, sliding window, rotate array, matrix traversal
P3 — Medium	Adjacent topic supported by broader reports	Stock profit, anagram grouping, rain water, basic DP
P4 — Lower	Useful only after core coverage	Trees, graphs, harder DP

2. Reported Questions — Drive A (17 Dec 2025, Sitapura Jaipur)

A detailed Associate Software Engineer report describes two technical rounds. Round 1 was 30 minutes with 3 questions, where candidates could attempt 2/3; Round 2 was 90 minutes with 7 questions, where candidates could attempt 5/7. The report emphasizes DSA, logic building and coding without built-in functions.

A1. Stable segregation — [-13, 5, -12, 6, -5, 7, 8, -9] → [-13, -12, -5, -9, 5, 6, 7, 8]. Preserve relative order.

Expected approach: Stable partition; can be O(n) with extra space, or optimized variants depending on constraints.

A2. Reverse string while keeping special characters fixed — "ab-cd" → "dc-ab".

Expected approach: Two pointers over the string; move past non-alphabetic/special positions.

A3. Binary pattern — Generate: 1 / 10 / 101 / 1010 / 10101.

Expected approach: Nested loops; parity of column controls 0/1.

A4. First non-repeated character — "swiss" → w.

Expected approach: Frequency count + second pass; O(n).

A5. Contiguous subarray with target sum — [2,4,5,6,3,8], target 15 → indices 1..3.

Expected approach: Sliding window for positive numbers; prefix sum + map for general integers.

A6. Longest palindromic substring — "babad" → "bab" or "aba".

Expected approach: Expand-around-center $O(n^2)$ is highly suitable; DP is another route.

A7. Decimal to binary without built-ins — Convert 5 to binary using arithmetic rather than library conversion.

Expected approach: Repeated division by 2; collect remainders.

A8. Snake pattern — 1 2 3 4 / 8 7 6 5 / 9 10 11 12.

Expected approach: 2D traversal with alternating row direction.

A9. Fibonacci triangle/pattern — Generate Fibonacci sequence in the reported triangular/rotated arrangement.

Expected approach: Maintain previous two values; separate sequence generation from printing.

A10. Equilibrium point — [1,3,5,2,2] → index 2 (value 5).

Expected approach: Total sum + running left sum gives O(n) time and O(1) extra space.

3. Reported Questions — Drive B (06 Jun 2026)

A June 2026 Associate Software Developer report describes two technical rounds. Round 1 lasted 30 minutes and included string compression, merging sorted arrays, and finding a character from a compressed representation. Round 2 lasted 120 minutes and focused on implementation, optimization, and explaining the approach.

B1. String compression — "aaaabbbccccc" → "a4bbbc5" (single occurrence remains unnumbered).

Expected approach: Run-length encoding with a count per run.

B2. Merge two sorted arrays — [3,8,11,56,75] + [13,25,56,65,85,94] → sorted merged array.

Expected approach: Two pointers; O(n+m).

B3. K-th character from compressed string — "a10b5c3", K=16 → c.

Expected approach: Parse counts and locate the block containing K without expanding the string.

B4. Concentric pattern — Generate the reported A/1/2 concentric pattern for n=5.

Expected approach: Nested loops; derive the value from distance to the nearest boundary.

B5. Spiral matrix — Generate the reported 4×4 spiral matrix: 0 1 2 3 / 11 12 13 4 / 10 15 14 5 / 9 8 7 6.

Expected approach: Maintain top, bottom, left, right boundaries.

B6. Conditional even-odd adjacent swap — [2,5,7,8,4,5] → [5,2,7,8,5,4].

Expected approach: Scan adjacent pairs and swap when the reported condition is satisfied.

B7. Two Sum — [2,6,15,13,8], target 21 → [1,2].

Expected approach: HashMap gives O(n) expected time.

B8. Pangram check — Determine whether a sentence contains every English alphabet letter.

Expected approach: Boolean/frequency array of size 26; normalize case.

B9. Number-star pattern — Generate the reported increasing number/star pattern for 5 rows.

Expected approach: Nested loops and a deterministic rule for alternating number/star output.

B10. Even divisor sum minus odd divisor sum — For 12, divisors are 1,2,3,4,6; reported result is 8.

Expected approach: Enumerate divisors up to sqrt(n), adding complementary divisors carefully.

4. Additional Recent Cyntexa Evidence

June 2026 Jaipur: A Glassdoor report says one Associate Software Engineer process had four rounds: three DSA rounds (basic, medium, hard) followed by HR. This shows that the number of rounds can vary by hiring cycle/role.

June 2026 Jaipur: Another report describes Phase 1 as two coding rounds: basic string/array questions, then an advanced round with string/array/character manipulation, patterns and 2D matrices at roughly easy-medium/medium level.

Aug–Sep 2025 Jaipur: Reports mention pen-and-paper DSA, rotate-array, transpose, pattern questions, followed by a harder round involving string manipulation and the water-holding/rain-water style problem.

June 2025 Durgapura: A report mentions arrays, strings, patterns, trees, then medium-hard questions including DP, hashing/anagram grouping; reported examples include reverse array, group anagrams and transpose matrix.

Apr 2025: A trainee software developer report says the first coding round focused on arrays, strings, matrices and some trees, while the second emphasized general problem-solving rather than one particular data structure.

2025 Reddit Jaipur: A candidate who reported being selected described sentence-palindrome logic, maximum path sum across two sorted arrays, minimum blocks to reach a target, and a word-chain problem based on matching the last two characters to the next word's first two.

Key conclusion: the exact question set is not fixed. However, the recurring center of gravity is unusually consistent: strings, arrays, matrices/patterns, basic hashing, and medium-level logic/optimization.

5. 100-Question Cyntexa Preparation Set

The following set is deliberately built around the observed Cyntexa question families. P1 questions should be mastered first; P2 should be practiced until you can solve them quickly; P3 is insurance for harder variants.

String

#	Priority	Practice question	Core technique
1	P1	Reverse a string in-place / without built-in reverse.	Two pointers.
2	P1	Check whether a string is a palindrome.	Two pointers.
3	P1	Check whether a sentence is a palindrome after ignoring non-letters and normalizing case.	Two pointers + character filtering.
4	P1	Find the first non-repeating character.	Frequency array/HashMap + second pass.
5	P1	Run-length encode a string: aaaabbbccccc → a4b3c5; handle count=1 as specified.	Run scanning.
6	P1	Given run-length encoded text such as a10b5c3, find the K-th expanded character without expanding.	Parse blocks + cumulative counts.
7	P1	Check whether two strings are anagrams.	Frequency counting.
8	P1	Group anagrams from an array of strings.	Canonical key / frequency signature.
9	P1	Find the longest common prefix of a list of strings.	Column scan or progressive prefix.
10	P1	Check whether a string is a pangram.	26-element boolean array.
11	P2	Remove duplicate characters while preserving first occurrence.	Seen set/boolean array.
12	P2	Count vowels, consonants, digits and special characters.	Single scan.
13	P2	Reverse only alphabetic characters while keeping special characters fixed.	Two pointers.
14	P2	Reverse every word in a sentence while preserving word order.	Token/character manipulation.
15	P2	Find the most frequent character; resolve ties by first appearance.	Frequency + first-index tracking.
16	P2	Check whether one string is a rotation of another without using a built-in search routine.	Length + manual substring search.
17	P2	Find the longest palindromic substring.	Expand around center.
18	P2	Count palindromic substrings.	Expand around center.
19	P2	Find the longest substring without repeating characters.	Sliding window + last index.
20	P2	Find the longest substring with at most K distinct characters.	Sliding window + frequency map.
21	P2	Compress a string using counts and compare compressed length with original.	Run-length encoding.
22	P2	Validate balanced parentheses/brackets in a string.	Stack.
23	P3	Find the minimum window containing all characters of another string.	Sliding window.
24	P3	Implement substring search without built-in contains/indexOf.	Naive scan; optionally KMP.
25	P3	Determine whether two strings are isomorphic.	Two-way mapping.

Array

#	Priority	Practice question	Core technique
26	P1	Merge two sorted arrays.	Two pointers.
27	P1	Two Sum: return indices for a target.	HashMap.
28	P1	Stablely move all negative values before positives while preserving order.	Stable partition.
29	P1	Find an equilibrium index where left sum equals right sum.	Total sum + running sum.
30	P1	Find a contiguous subarray with a target sum for positive numbers.	Sliding window.
31	P1	Reverse an array without built-in reverse.	Two pointers.
32	P1	Rotate an array by K positions.	Reversal algorithm.
33	P1	Find maximum and minimum in one pass.	Single scan.
34	P1	Remove duplicates from a sorted array in-place.	Two pointers.
35	P1	Move all zeroes to the end while preserving non-zero order.	Two pointers.
36	P2	Find the second largest distinct element.	One/two-pass.
37	P2	Find the missing number from 0..n.	XOR or sum.
38	P2	Find the duplicate number when values are in 1..n.	Floyd or frequency.

#	Priority	Practice question	Core technique
39	P2	Find all duplicate values.	Frequency/counting.
40	P2	Find intersection of two arrays.	Set/frequency or sorted two pointers.
41	P2	Find union of two sorted arrays without duplicates.	Two pointers.
42	P2	Merge overlapping intervals.	Sort + greedy scan.
43	P2	Find maximum subarray sum.	Kadane's algorithm.
44	P2	Find a pair with a given difference.	HashSet/two pointers after sorting.
45	P2	Find three numbers whose sum equals target.	Sort + two pointers.
46	P2	Find majority element.	Boyer-Moore voting.
47	P2	Find leaders in an array.	Right-to-left maximum.
48	P2	Find the longest consecutive sequence.	HashSet.
49	P2	Product of array except self.	Prefix/suffix products.
50	P2	Sort an array containing only 0, 1 and 2.	Dutch National Flag.
51	P2	Find a subarray with maximum product.	Track max/min products.
52	P2	Find subarrays whose sum equals K for general integers.	Prefix sum + HashMap.
53	P3	Best Time to Buy and Sell Stock for one transaction.	Running minimum + max profit.
54	P3	Best Time to Buy and Sell Stock II for unlimited transactions.	Greedy accumulation.
55	P3	Trapping Rain Water.	Two pointers or prefix maxima.

Matrix/Pattern

#	Priority	Practice question	Core technique
56	P1	Transpose a matrix.	Swap indices / construct result.
57	P1	Generate a spiral matrix.	Boundary traversal.
58	P1	Print a matrix in spiral order.	Boundary traversal.
59	P1	Generate a snake matrix where alternate rows reverse direction.	Row parity.
60	P1	Print binary pattern: 1 / 10 / 101 / 1010 / ...	Nested loops + parity.
61	P1	Generate increasing number/star pattern.	Nested loops.
62	P1	Generate Fibonacci triangle/pattern.	Sequence + nested printing.
63	P1	Generate a concentric A/1/2 pattern for N.	Distance-to-boundary logic.
64	P2	Rotate a square matrix 90° clockwise.	Transpose + reverse rows.
65	P2	Rotate a square matrix 90° anticlockwise.	Transpose + reverse columns.
66	P2	Search a value in a row/column sorted matrix.	Staircase search.
67	P2	Set matrix rows/columns to zero when a cell is zero.	Marker rows/columns.
68	P2	Find diagonal sums/difference in a square matrix.	Index rules.
69	P2	Print boundary elements of a matrix.	Boundary loops.
70	P2	Print matrix diagonally.	Diagonal traversal.
71	P2	Find largest row sum and largest column sum.	Two scans.
72	P2	Print a hollow square/rectangle pattern.	Boundary condition.
73	P2	Print Pascal's triangle.	Combinations / iterative row construction.
74	P2	Print a centered pyramid/diamond pattern.	Spaces + symbols.
75	P3	Find the largest rectangle of 1s in a binary matrix.	Histogram + stack.

Logic/Math

#	Priority	Practice question	Core technique
76	P1	Compute sum of even divisors minus sum of odd divisors.	Enumerate divisors up to sqrt(n).
77	P1	Convert decimal to binary without built-in conversion.	Repeated division by 2.

#	Priority	Practice question	Core technique
78	P2	Check whether a number is prime.	Trial division to sqrt(n).
79	P2	Print all prime numbers up to N.	Sieve or repeated primality.
80	P2	Find GCD and LCM.	Euclidean algorithm.
81	P2	Reverse an integer and detect overflow.	Digit extraction.
82	P2	Check Armstrong number.	Digit powers.
83	P2	Check palindrome number.	Reverse digits.
84	P2	Count set bits in an integer.	Brian Kernighan.
85	P2	Generate Fibonacci numbers iteratively.	Two variables.
86	P2	Find the K-th Fibonacci number.	Iterative / matrix optional.
87	P3	Count divisors of N efficiently.	Factor pairs up to sqrt(n).
88	P3	Find whether N can be represented as a sum of two squares.	Two pointers / number theory.

General DSA

#	Priority	Practice question	Core technique
89	P1	Find the maximum path sum across two sorted arrays, switching at common values.	Two-pointer traversal with segment sums.
90	P1	Find the longest word chain where the next word starts with the previous word's last two characters.	Graph/DP over word transitions.
91	P2	Minimum number of blocks/coins needed to reach a target.	Greedy if canonical; otherwise DP.
92	P2	Implement a stack using an array.	Top pointer.
93	P2	Implement a queue using an array.	Front/rear pointers.
94	P2	Find next greater element for every array value.	Monotonic stack.
95	P2	Evaluate a postfix expression.	Stack.
96	P2	Remove adjacent duplicate characters repeatedly.	Stack.
97	P3	Detect a cycle in a linked list.	Floyd two pointers.
98	P3	Reverse a linked list.	Iterative pointer reversal.
99	P3	Maximum depth of a binary tree.	DFS recursion/BFS.
100	P3	Climbing stairs / minimum-cost climbing stairs.	1D DP.

6. Detailed Coding-Round Analysis

Round 1 — How to approach it

Typical reported shape: ~30 minutes, roughly 3 questions, with candidates expected to solve around 2 or more. The highest-value skills are fast implementation, reading the exact output requirement, and avoiding unnecessary library shortcuts.

Time allocation for 3 questions: 2–3 minutes read/plan → 7–10 minutes each for the first two → use the remainder for the third and testing. If one question becomes a time sink after ~8–10 minutes, move on.

Likely Round-1 families: reverse/palindrome, frequency, compression, merge, stable partition, basic array manipulation, simple pattern, decimal/binary conversion.

Round 2 — How to approach it

Typical reported shape: 90–120 minutes, with around 7 questions in one recent report and a smaller set in other cycles. The round is more about breadth plus optimization: you may see matrices/patterns alongside medium array/string problems.

Best order: solve obvious implementation questions first; then HashMap/two-pointer/sliding-window problems; then matrix/pattern questions; leave the most uncertain DP/graph problem for last.

What the evaluator appears to care about

• Correctness on edge cases.

• Ability to explain the idea before coding.

• Time and space complexity.

• Avoiding unnecessary built-in shortcuts when the test asks for manual implementation.

• Readable variable names and clean control flow.

• Being able to optimize a brute-force solution when asked.

7. Highest-Yield Topic Ranking

Rank	Topic	Why it matters	Target
1	Strings	Repeated directly across recent rounds.	Master 20–25 problems
2	Arrays	Repeated directly and combined with hashing/two pointers.	Master 20–25 problems
3	Patterns + matrices	Explicitly reported in both recent detailed drives.	Master 15–20 problems
4	HashMap/frequency	Appears in Two Sum, non-repeat, anagrams and optimization.	Master 8–10 patterns
5	Two pointers / sliding window	Natural solution family for palindrome, subarray and string questions.	Master 8–10 problems
6	Basic math	Divisors, binary conversion and number logic recur.	Master 8 problems
7	Stack/Linked List	Less frequent but useful for medium variants.	5–8 problems
8	DP/Graphs/Trees	Reported in some cycles, especially harder rounds, but less central to the two reported drives.	Basics + selected mediums

8. 7-Day Intensive Preparation Plan

Day	Focus	Work
Day 1	Strings	Questions 1–10, 13–16. Focus on two pointers, frequency and parsing.
Day 2	Arrays	Questions 26–40. Focus on merge, stable partition, rotation, duplicates, Two Sum.
Day 3	Arrays + medium	Questions 41–55. Kadane, 3Sum, majority, consecutive, prefix sums, stock, rain water.
Day 4	Matrices + patterns	Questions 56–75. Transpose, spiral, snake, concentric, rotations, zero matrix, patterns.
Day 5	Math + implementation	Questions 76–88. Divisors, binary, primes, GCD, number logic, Fibonacci.
Day 6	Mixed timed mocks	Create two 30-minute Round-1 mocks and one 90–120-minute Round-2 mock.
Day 7	Revision	Redo every question you failed or took too long on. Practice explaining complexity aloud.

Mock rule: do not look at editorials while the timer is running. After the mock, classify each failure as syntax, logic, edge case, complexity, or time-management. This will tell you what to fix.

9. What NOT to Overprioritize

For the specific two-round campus format, do not let advanced topics crowd out the fundamentals. Reports do mention trees, graphs and DP in some Cyntexa cycles, but the repeated concrete questions are much more heavily weighted toward strings, arrays, matrices, patterns and general problem solving.

Still know the basics of:

- Linked-list reversal and cycle detection
- Stack/queue basics
- Tree traversal and height
- Simple 1D DP such as climbing stairs
- Basic graph traversal (BFS/DFS)

But if you have limited time, finish P1 strings/arrays/matrix questions before moving deeply into graphs or hard DP.

10. Final Exam-Day Checklist

- ■ Can I write two-pointer code without hesitation?
- ■ Can I count frequencies with an array/HashMap?
- ■ Can I merge two sorted arrays manually?
- ■ Can I reverse while preserving special-character positions?
- ■ Can I solve equilibrium index in O(n)?
- ■ Can I solve positive-number target subarray with a sliding window?
- ■ Can I explain longest palindrome with expand-around-center?
- ■ Can I parse run-length encoded strings without expanding them?
- ■ Can I generate spiral/snake/transpose matrix logic?
- ■ Can I print common star/number patterns quickly?
- ■ Can I convert decimal to binary without built-ins?
- ■ Can I state time and space complexity for every solution?
- ■ Can I test empty input, one element, duplicates, negatives and boundary cases?

11. Source Notes

The evidence in this document was gathered from publicly available interview reports. Reported questions are reproduced only as concise problem summaries/examples; the 100-question set is original preparation material derived from the recurring topic patterns.

Primary recent sources:

1. Harun Sheikh — Associate Software Developer, Cyntexa, 06/06/2026 — detailed two-round report.
2. Harun Sheikh — Associate Software Engineer, Cyntexa Sitapura Jaipur, 17/12/2025 — detailed two-round report.
3. Glassdoor — Cyntexa interview reports, including June 2026 Jaipur and Aug–Sep 2025 Jaipur reports.
4. Reddit — Manipal Jaipur discussion containing a December 2025 Cyntexa candidate's reported questions.
5. Additional Glassdoor reports on arrays, strings, matrices, DP, hashing/anagram grouping and general problem solving.

Important caveat: Candidate reports are self-reported and may contain transcription/output mistakes. For example, one reported decimal-to-binary output is internally inconsistent. The preparation strategy therefore uses the underlying problem type rather than treating every copied output as authoritative.